

The Indelible Federation: A Federated SPV Relay Mesh for Bitcoin SV

zcool

February 2026 (Revised: April 19, 2026 — v5: P2P peer retention research and four protocol-level fixes informed by reading the BSV node C++ source code alongside bridge JavaScript; inbound P2P listening on port 8333; novel transaction relay with immediate `inv` forwarding and shared tx cache; good peer persistence; DNS seed crawler and `seed.indelible.one` ; BCH blacklist; x402 HTTP 402-based payment middleware; mesh WebSocket port moved from 8333 to 18333; systemd integration; deployment expanded to seven geographically distributed bridges; Wood Wide Web framing — bridges as mycelium, apps as trees, transactions as nutrients. v4: data relay layer. v3: indexing layer. v2: on-chain registry, stake bonds, peer scoring, cryptographic handshake, gossip protocol, protocol parsing.)

Abstract

This paper presents a federated architecture for Simplified Payment Verification (SPV) relay nodes operating on the Bitcoin SV network. The system addresses the fundamental problem facing BSV application developers: the choice between running a full node (expensive, complex) and depending on centralised third-party APIs (single point of failure, someone else's uptime). The relay mesh provides a middle ground -- lightweight bridge nodes that peer with each other over an open, decentralised protocol, using the BSV blockchain itself for peer discovery via an on-chain CBOR registry.

The architecture comprises: (1) a P2P layer speaking native Bitcoin protocol version 70016 over TCP; (2) an SPV client layer managing peer connections, transaction broadcast via `inv` / `getdata` / `tx` , and transaction lookup via `getdata MSG_TX` ; (3) a header synchronisation layer downloading and storing all block headers in a dual-indexed LevelDB database; (4) an API layer serving client applications through REST and WebSocket interfaces; (5) a federation layer providing bridge-to-bridge peering with cryptographic identity, on-chain registration, stake bonds, and local peer scoring; (6) a supervision layer spanning all modules for crash resilience and self-healing; (7) an indexing layer providing transaction confirmation tracking, Merkle proof storage, BSV-20 token accounting, content-addressed inscription storage, and historical backfill; and (8) a data relay layer propagating ephemeral signed data envelopes -- topic-routed, TTL-bounded, and payload-opaque -- across the mesh via gossip, enabling applications to distribute real-time signals without on-chain transactions.

Bridge identity is tied to a BSV keypair. Registration is published on-chain via CBOR-encoded `OP_RETURN` transactions. Stake bonds lock satoshis to the bridge operator's own address as proof of BSV ownership. Peer scoring -- computed locally by each bridge with no central authority -- weights data accuracy (40%), uptime (30%), response time (20%), and stake age (10%). Peers below a threshold score are auto-disconnected.

The system operates in production across seven geographically distributed bridges (Dallas, New Jersey, Chicago, Atlanta, Silicon Valley, and two additional nodes), deployed for the Indelible platform (`indelible.one`) as a reference implementation. Each bridge connects to 15-25 Bitcoin full nodes via native P2P peer exchange (`getaddr` / `addr`) and accepts inbound connections from BSV nodes on port 8333, maintains header chain synchronisation across 945,000+ blocks, and processes transaction broadcasts without dependence on any third-party API services. The protocol is open -- any developer can run a bridge, register on-chain, and join the mesh.

The architecture mirrors a pattern 1.5 billion years old. Underground fungal networks called mycelium connect every tree in a forest -- routing nutrients, sharing resources, self-healing when threads are severed. Bridges are the mycelium. Applications are the trees. Transactions are the nutrients. The federation is a Wood Wide Web for Bitcoin.

1. Introduction

Satoshi Nakamoto described Simplified Payment Verification in Section 8 of the Bitcoin whitepaper [1] as a method by which a node "can verify payments without running a full network node" by keeping "a copy of the block headers of the longest proof-of-work

chain" and obtaining "the Merkle branch linking the transaction to the block it's timestamped in." This design permits lightweight clients to verify transaction inclusion with mathematical certainty without storing the full blockchain.

In practice, however, most SPV implementations operate as isolated clients. Each connects independently to one or a small number of Bitcoin full nodes, downloads its own copy of the header chain, and has no awareness of other SPV infrastructure. If the connected full node does not have a particular transaction in its mempool, the SPV client has no recourse. If the full node is slow or temporarily unreachable, the client experiences degraded service with no failover path.

This isolation is a material problem for any developer building on BSV. Every blockchain ecosystem faces the same tooling gap: Ethereum has Infura and Alchemy; Solana has Helius and QuickNode. These services provide API access to blockchain networks without running a full node. On BSV, the options are sparse. Developers either run a full node (requiring significant disk, bandwidth, and operational overhead) or depend on centralised APIs with opaque uptime guarantees and rate limits they do not control.

The relay mesh fills this gap. It provides BSV developers with a third option: run a lightweight bridge node that peers with other bridges, syncs headers, and relays transactions -- all without storing the full blockchain. The protocol is open. Any developer can run a bridge, register it on-chain with a stake bond, and join the mesh. Discovery is decentralised: bridges find each other by scanning the BSV blockchain for registry transactions. Trust is local: each bridge scores its peers independently based on observed behaviour, with no central reputation authority.

The system was designed and first deployed for the Indelible platform (indelible.one), which stores AI conversation memory, encrypted files, and project archives permanently on the Bitcoin SV blockchain. The relay mesh proved that federated SPV infrastructure can serve production blockchain applications with sub-second latency and zero third-party dependencies.

The architecture converges on patterns found in nature. Underground fungal networks called mycelium connect every tree in a forest — routing nutrients, sharing resources, self-healing when threads are severed. The federation mirrors this structure: bridges are mycelium threads, applications are trees that tap into the mesh, and transactions are the nutrients flowing through the network. This convergence is not aesthetic — it is structural. Distributed resource sharing with no central authority, evolved over 1.5 billion years in forests, produces the same topology when implemented in software. Section 19 develops this parallel in detail.

This paper describes the architecture, protocol, and production deployment as a reference for the broader BSV developer community.

2. Background

2.1 Simplified Payment Verification

The Bitcoin whitepaper [1] describes SPV in Section 8:

"It is possible to verify payments without running a full network node. A user only needs to keep a copy of the block headers of the longest proof-of-work chain, which he can get by querying network nodes until he's convinced he has the longest chain, and obtain the Merkle branch linking the transaction to the block it's timestamped in."

This establishes two core requirements for SPV: maintenance of the header chain and the ability to verify Merkle proofs. Section 7 [1] explains the data structure that makes this possible -- the Merkle tree. Each block header contains a Merkle root that commits to every transaction in the block. A Merkle branch (or proof) is a logarithmic-size path from a leaf transaction to the root, allowing verification without downloading the full block.

Section 2 [1] defines the transaction model itself: "We define an electronic coin as a chain of digital signatures." Each transaction takes inputs from previous transaction outputs and produces new outputs, forming a directed acyclic graph of ownership.

2.2 The Broadcast Protocol

Section 5 [1] specifies the network protocol: "New transactions are broadcast to all nodes." In the Bitcoin peer-to-peer protocol, this broadcast follows a specific sequence. A node announces it possesses a transaction by sending an `inv` (inventory) message containing the transaction identifier. Interested peers respond with a `getdata` message requesting the full transaction. The originating node then sends the complete `tx` message.

This three-step `inv / getdata / tx` handshake is essential. Earlier iterations of the relay system attempted to push raw `tx` messages directly to peers without the preceding `inv` announcement. Bitcoin full nodes consistently ignored these unsolicited transactions. The protocol requires the announcement-request-delivery flow; any deviation results in transactions never reaching the mining network.

2.3 The NODE_BLOOM Problem

BIP37 [2] introduced bloom filters as a mechanism for SPV clients to receive only transactions matching a particular pattern. An SPV client would send a `filterload` message containing a bloom filter, and the connected full node would apply this filter to incoming transactions and blocks, sending only matches.

On Bitcoin SV, `NODE_BLOOM` is disabled by default (`DEFAULT_PEERBLOOMFILTERS=false`). Sending a `filterload` message to a BSV full node triggers an immediate Misbehaving penalty of 100, resulting in a 24-hour ban from that peer. This is not a bug; it is a deliberate design decision reflecting the view that bloom filters introduce privacy leaks and impose computational burden on full nodes.

This means that the entire class of SPV implementations relying on bloom filters is inoperable on BSV. An alternative approach is required: direct `getdata` requests using `MSG_TX` (type 1) to fetch specific transactions by their identifier.

2.4 Overlay Networks

Wright's paper on overlay network architectures [3] (Section 3, "Network Architecture") describes a structured approach to node-to-node communication layered above the base Bitcoin protocol. The paper details Pastry DHT routing for efficient message delivery, reputation scoring for peer quality assessment, formal node admission protocols, the S-net overlay for structured communication, and mechanisms for eclipse attack resistance.

This work provided the conceptual blueprint for the federation layer described in this paper. The current implementation realises several of the mechanisms Wright described: reputation scoring (Section 11.7), node admission via on-chain registration and stake bonds (Sections 7.2-7.3), and cryptographic identity verification (Section 7.5). The system uses a gossip-based peer discovery model rather than full Pastry DHT routing, with DHT routing deferred as future work for larger mesh sizes.

3. Architecture Overview

The system is organised into eight distinct layers, each with a dedicated module, protocol, and purpose.

Layer	Module	Protocol	Port	Purpose
P2P	<code>bsv-peer.js</code>	TCP (Bitcoin)	8333	Wire-level Bitcoin protocol: binary message framing, checksum validation, handshake. Both outbound and inbound connections.
SPV Client	<code>bsv-node-client.js</code>	TCP (Bitcoin)	8333	Peer management, transaction broadcast via <code>inv/getdata/tx</code> , transaction lookup, novel tx relay, good peer persistence
Header Sync	(integrated)	TCP (Bitcoin)	8333	Block header download, LevelDB dual-index storage (by height and hash)
Indexing	<code>persistent-store.js</code>	LevelDB	—	Tx confirmation tracking, Merkle proofs, BSV-20 tokens, CAS inscription storage, backfill
API	<code>status-server.js</code>	HTTP, WebSocket	9333	REST API, WebSocket interface, operator dashboard, x402 payment middleware
Data Relay	<code>data-relay.js</code> , <code>data-endpoints.js</code>	WebSocket, HTTP	18333, 9333	Ephemeral signed data envelopes: topic-filtered gossip, TTL-bounded storage, pull-based catch-

				up
Federation	handshake.js, gossip.js, peer-scorer.js, registry	WebSocket, BSV OP_RETURN	18333	On-chain registration, cryptographic peering, gossip discovery, local peer scoring
Supervision	(integrated)	—	—	Crash resilience, self-healing, systemd integration

3.1 Port Layout

The bridge uses three ports, each serving a distinct protocol layer:

Port	Protocol	Purpose
8333	TCP (Bitcoin P2P)	Outbound connections to BSV full nodes and inbound connections from BSV nodes. Standard Bitcoin port — BSV nodes expect peers here.
18333	WebSocket	Bridge-to-bridge mesh peering: cryptographic handshake, gossip, transaction relay, header sharing, data envelopes, session sync.
9333	HTTP	REST API for applications, operator dashboard, x402 payment endpoint.

Port 8333 was originally shared between BSV P2P and federation mesh WebSocket. This created a conflict: BSV nodes connecting inbound expected Bitcoin protocol, but the port was serving WebSocket. The port split to 18333 for mesh and 8333 exclusively for Bitcoin P2P resolved this and enabled inbound P2P listening (Section 4.3).

The layers are ordered by proximity to the Bitcoin network. The P2P layer handles the raw wire protocol. The SPV client manages peer connections and transaction operations. The header sync maintains the complete chain of block headers. The indexing layer tracks transaction lifecycle, stores Merkle proofs, indexes BSV-20 tokens, and manages content-addressed inscription storage. The data relay layer propagates ephemeral signed data envelopes between bridges via topic-filtered gossip. The API layer faces outward to application clients. The federation layer provides bridge-to-bridge communication, identity, and trust. The supervision layer ensures crash recovery.

Each layer is self-contained. A bridge can operate without the federation layer -- it simply has no peers and no mesh resilience. The federation layer can operate without the API layer -- bridges peer and relay transactions without serving external clients. This modularity ensures that partial failures do not cascade.

The architecture is designed so any developer can run a bridge. The system is packaged as three npm packages (`@relay-federation/common` , `@relay-federation/registry` , `@relay-federation/bridge`) and installs with a single command: `npm install -g @relay-federation/bridge` .

4. P2P Layer

The P2P layer (`bsv-peer.js`) implements the Bitcoin peer-to-peer protocol at the binary wire level.

4.1 Protocol Implementation

Messages are framed with the BSV mainnet magic bytes (`0xe3e1f3e8`), followed by a 12-byte null-padded command string, a 4-byte little-endian payload length, a 4-byte checksum (the first four bytes of the double-SHA256 of the payload), and the payload itself.

The system uses protocol version 70016, the current BSV protocol version that includes support for large messages, and identifies itself with user agent `/Indelible Bridge:0.3.60/` . Testing confirmed that user agent string does not affect peer retention — BSV nodes do not discriminate based on user agent during eviction decisions (Section 5.6).

4.1.1 Services Bitmask: Why Bridges Must Not Claim NODE_NETWORK

The `version` message includes a `services` bitmask. Setting `services=1n` claims `NODE_NETWORK` — advertising that this node stores the full blockchain and can serve arbitrary blocks on request. BSV full nodes expect `NODE_NETWORK` peers to respond to `getdata MSG_BLOCK` requests. When a bridge claiming `NODE_NETWORK` cannot serve blocks, the requesting node marks it as misbehaving.

The bridge sets `services=0n` (`NODE_NONE`), correctly advertising that it is a pure SPV client. This is not a limitation but a protocol-level truth: the bridge syncs headers and relays transactions, but does not store or serve full blocks. Setting the honest value eliminates an entire class of disconnection caused by BSV nodes discovering the lie.

4.2 Connection Handshake

Connection establishment follows the Bitcoin protocol handshake:

1. The bridge opens a TCP connection to a full node on port 8333.
2. The bridge sends a `version` message containing its protocol version, services bitmask (set to `0n`, `NODE_NONE`), timestamp, and **current block height** — the bridge's actual synced header height, not a stale value. Advertising a height thousands of blocks behind the chain tip causes immediate deprioritisation by full nodes (see Section 5.6, Fix 1).
3. The full node responds with its own `version` message, including its best block height.
4. The bridge sends a `verack` (version acknowledgement) message.
5. The bridge immediately sends a `protoconf` message advertising its maximum receive payload size (2MB), as required by protocol version 70016 and above.
6. The full node may send `authch` (authentication challenge) messages related to MinerID. These are ignored silently. Per the BSV node source code, connections proceed even if authentication is not completed. Non-mining SPV clients have no MinerID key and cannot sign authentication challenges.

4.3 Inbound P2P Listening

Bridges accept inbound TCP connections from BSV full nodes on port 8333. When an inbound connection arrives, the bridge waits for the remote node's `version` message, validates it (rejecting non-BSV clients — see Section 5.7), sends its own `version` and `verack`, then promotes the connection to a full peer.

After establishing any outbound connection, the bridge sends a `sendSelfAddr()` message — an `addr` message containing its own public IP and port 8333. This advertises the bridge's existence to the BSV node's address manager (`addrman`), making the bridge discoverable by other BSV nodes via `getaddr` / `addr` peer exchange. Over time, the bridge's address propagates through the BSV network's gossip layer, attracting additional inbound connections without active solicitation.

This transforms bridges from passive consumers to active participants in the BSV network topology. A bridge that both connects outbound and accepts inbound occupies a structural position in the network graph — closer to a peer than a client.

4.3 Message Handling

The P2P layer handles all standard Bitcoin protocol messages: `version`, `verack`, `ping` / `pong`, `headers`, `inv`, `tx`, `notfound`, `getdata`, `reject`, `merkleblock`, `protoconf`, and `authch`. Unrecognised messages are logged and ignored. The layer emits events for each message type, allowing upper layers to register handlers without coupling to the wire protocol.

5. SPV Client Layer

The SPV client layer (`bsv-node-client.js`) manages peer connections and transaction operations. It consumes events from the P2P layer and orchestrates multi-peer communication.

5.1 Peer Discovery

Peers are discovered through four mechanisms that operate in sequence:

1. **Good peer persistence:** On startup, the bridge loads `good-peers.json` from its data directory. These are peers that proved reliable in previous sessions — specifically, peers that sent valid block headers, confirming they speak BSV protocol and maintain current chain state. Good peers are tried first, before any other discovery mechanism, giving the bridge a warm start after restarts.
2. **DNS Seeds:** The bridge resolves DNS seed addresses from `seed.bitcoinsv.io`, `seed.satoshisvision.network`, `seed.cascharia.com`, and `seed.indelible.one` (the federation's own DNS seed, populated by the crawler described in Section 5.8).

The state of BSV's legacy DNS seed infrastructure as of April 2026:

Seed	Status
<code>seed.bitcoinsv.io</code>	3 IPv4 addresses
<code>dnsseed.bitcoinsv.io</code>	Dead — domain does not exist
<code>seed.cascharia.com</code>	Resolves, returns nothing
<code>dnsseed.cascharia.com</code>	Resolves, returns nothing
<code>seed.satoshisvision.network</code>	2 IPv4 addresses
<code>dnsseed.satoshisvision.network</code>	Dead — domain does not exist
<code>seed.indelible.one</code>	26+ verified-alive peers (federation crawler)

Five peers from six legacy seeds. Two domains no longer exist. Two return empty responses. This is the DNS seed crisis that prompted the federation to build its own crawler and DNS seed (Section 5.8) — the network's reproductive system. The federation seed at `seed.indelible.one` provides 26+ verified-alive peers.

3. **Native P2P Peer Exchange (`getaddr` / `addr`):** After completing the handshake with each peer, the bridge sends a `getaddr` message -- the Bitcoin protocol's built-in peer discovery mechanism. Each connected node responds with an `addr` message containing the IP addresses and ports of all nodes in its address book. The bridge parses these `addr` messages (30-byte entries: 4 bytes timestamp, 8 bytes services, 16 bytes IPv4-mapped IPv6 address, 2 bytes port), filters for IPv4 nodes on port 8333, and connects to each new peer. Those new peers also respond to `getaddr` with their own address books, creating a cascading discovery effect.
4. **On-chain beacon backfill:** On startup, each bridge scans the beacon address (`1KhH4VshyN8PnzxbTSjiojcQbbABNSZyZr`) for registration transactions. This discovers all bridges that registered before this bridge came online. Combined with gossip propagation of new registrations, this provides zero-configuration mesh discovery.

The four-stage approach — warm start from persisted good peers, DNS seeds (including the federation's own), native P2P peer exchange, and on-chain beacon discovery — eliminates all third-party API dependencies for peer discovery. If every legacy DNS seed dies tomorrow, the federation continues discovering peers through its own seed, its own crawler, persisted good peers, and the on-chain registry.

5.1.1 Stagger Connect

Connecting to many peers simultaneously triggers rate-limiting behaviour in BSV nodes. The bridge connects in batches of 4 peers with 2-second delays between batches. This stagger applies to both initial startup connection and periodic maintenance cycles.

5.2 Transaction Broadcasting

Transaction broadcast follows the `inv` / `getdata` / `tx` flow described in Section 5 of the Bitcoin whitepaper [1]. The SPV client:

1. Computes the transaction identifier (double-SHA256 of the raw bytes, reversed).

2. Stores the raw transaction in a pending broadcasts map.
3. Sends an `inv` message announcing the transaction to all connected peers.
4. When peers respond with `getdata`, the P2P layer serves the full `tx` message.

This procedure is repeated to all connected peers. In the current deployment, transactions are broadcast to 250+ peers simultaneously, reaching the vast majority of the reachable BSV network.

5.3 Transaction Fetching

Transactions are fetched using `getdata` with inventory type `MSG_TX` (1). The SPV client constructs an inventory vector containing the desired transaction identifier, sends the `getdata` message, and waits for the corresponding `tx` response. If the remote peer does not possess the transaction, it responds with a `notfound` message. The SPV client handles `notfound` explicitly, resolving the request as not-found immediately rather than waiting for a 10-second timeout.

5.4 Inventory Handling and Novel Transaction Relay

When a full node announces new transactions via `inv`, the bridge does two things simultaneously:

1. **Requests the full transaction** via `getdata MSG_TX` to obtain the raw bytes.
2. **Immediately forwards the `inv`** to all other connected BSV peers, before downloading the full transaction.

This immediate relay is critical for peer retention (Section 5.6). The bridge maintains a `seenTxids` Set for deduplication — each txid is tracked for 2 minutes before being pruned by a cleanup timer. When a peer subsequently requests the full transaction via `getdata`, the bridge serves it from a shared transaction cache (`_txCache`) accessible to all peer connections. This shared cache means any peer can serve a transaction that any other peer received — the cache is not per-connection.

Block inventory triggers a header re-synchronisation to update the local chain tip.

Cross-Protocol Relay: Mesh to P2P

Transactions arriving on the federation mesh (WebSocket, port 18333) are relayed to BSV peers (TCP, port 8333) using the same `inv / getdata / tx` flow. Without this cross-protocol relay, federation traffic generates zero novel relay credit with BSV nodes. The bridge treats mesh-originated transactions identically to P2P-originated ones for relay purposes.

5.5 Address Watching

The SPV client maintains a set of watched addresses. When a transaction arrives (via `getdata` response or peer announcement), the client decodes it and checks whether any output script contains the hash160 of a watched address. Matching transactions are stored in a local LevelDB and emit events to subscribed clients.

5.6 P2P Peer Retention: Reading the BSV Source Code

Early bridge deployments suffered from peer attrition — connections lasting minutes, not hours. Bridges maintained 0-3 peers and could not sustain reliable transaction relay. Understanding why required reading the BSV full node source code (C++, `bitcoin-sv-1.1.1`) alongside the bridge code (JavaScript) — two codebases in two languages solving the same problem from opposite sides of the same protocol.

The Eviction Algorithm

BSV full nodes decide which peers to keep using `AttemptToEvictConnection` in `src/net/net.cpp`. The algorithm protects peers across five layers:

Layer	Protected Peers	Criteria
Netgroup diversity	4	Different /16 subnets
Best ping	8	Lowest latency

Novel tx relay	4	Most recent <code>nLastTxTime</code> — only set when delivering a tx the node has never seen
Block relay	4	Most recent block delivery
Longest connected	50% of remaining	Uptime

Layer 3 is the critical one. `nLastTxTime` is set only when a peer delivers a transaction the full node has **never seen before**. The first relayer wins. Every subsequent relayer of the same transaction gets nothing. If a bridge never relays novel transactions, it has zero eviction protection in this layer and gets kicked within minutes.

The AddrMan Penalty System

`addrman.cpp` implements a separate reputation system for connection attempts:

- `GetChance()` applies a 99% penalty for peers retried within 10 minutes — $0.01 * nSinceLastTry$ scaling.
- 3 failed connection attempts mark a peer as "terrible" via `IsTerrible()`.
- Each failure applies an exponential 0.66x backoff (max 8 failures = 1/256 of normal selection chance).
- These penalties affect reconnection priority, not banning. A bridge that reconnects aggressively does not get banned — it gets deprioritised into near-zero selection probability.

Four Fixes

Armed with this understanding, four protocol-level fixes were implemented:

Fix 1 — Header sync before pack. The bridge advertised block height ~930,000 in its `version` message handshake. The BSV chain tip was ~945,500. Full nodes saw the bridge as 15,000 blocks behind — a stale node not worth keeping. The fix: connect to one peer first, sync headers to the current chain tip, then connect to remaining peers advertising the real height. Result: one bridge went from 3 peers to 20 within 10 minutes of deploying this fix.

Fix 2 — Immediate inv relay. When the bridge received an `inv` from a peer, it downloaded the full transaction before announcing to other peers. By then, another node had already relayed the same `inv`. The bridge lost the `nLastTxTime` race every time. The fix: forward the `inv` immediately, download the full tx in parallel. Store it in the shared tx cache so it can be served when `getdata` arrives from other peers. Result: novel relay acceptance went from 0% to 98%.

Fix 3 — Mesh-to-P2P relay. Transactions arriving on the federation mesh (WebSocket, port 18333) were logged but never forwarded to BSV peers (TCP, port 8333). The entire federation's transaction traffic generated zero novel relay credit. The fix: wire the mesh transaction handler to the P2P relay path. All mesh transactions are now relayed to BSV peers using the same `inv` / `getdata` / `tx` flow.

Fix 4 — Maintain cycle guard. The peer maintenance timer fired every 60 seconds, but each cycle took 73 seconds (145 addresses × 4/batch × 2s delay). Three concurrent cycles ran simultaneously — 435 connection attempts per minute. Combined with a bug where "good peers" bypassed all cooldown timers, this created a reconnection storm that triggered AddrMan's deprioritisation penalties. The fix: a concurrency guard allowing only one maintenance cycle at a time, base cooldown raised from 5 seconds to 120 seconds (exceeding the 60-second timer interval), max 20 connections per cycle, and demotion of good peers after 3 consecutive failures.

Results

Bridge	Peers Before	Peers After
Beta (NJ)	3	17-20
Alpha (Dallas)	0-2	12-18
Delta (Dallas)	0-2	3-20
Gamma (Chicago)	0	4

Epsilon (Atlanta)	0-4	4-20
Bridge-6 (SV)	2-3	3-5
Bridge-7 (crawler)	3-6	4-7

Novel relay hit rates: Beta 69%, Gamma 52%, Bridge-7 15% (fewer peers = fewer relay opportunities, but still contributing). The bridges went from passive consumers to active participants in the BSV transaction relay network.

5.7 BCH Blacklist

During peer discovery, the bridge occasionally connects to Bitcoin Cash (BCH) nodes operating on port 8333. BCH nodes send valid `version` messages but speak an incompatible protocol after the handshake — different block headers, different transaction validation rules, different chain.

The bridge validates inbound `version` messages by checking the user agent string for "Bitcoin SV". Connections from non-BSV nodes are rejected immediately after the `version` exchange. A race condition was discovered where BCH nodes send both `version` and `verack` in the same TCP packet; the buffer processing loop would process the `verack` after the disconnect had been initiated, bypassing the blacklist. Three guards were added to `bsv-peer.js` to handle this: a `destroyed` flag check before processing any message, a disconnect check before processing `verack`, and an explicit `socket.destroy()` call in the rejection path.

5.8 DNS Seed Crawler

One of the seven bridges (Bridge-7, Silicon Valley) runs a crawler that probes the BSV peer network. The crawler:

1. Maintains a database of 2,100+ known peer IP addresses gathered from `getaddr` responses across the fleet.
2. Every 5 minutes, attempts TCP handshakes with all known peers.
3. Categorises each peer by failure mode: alive, TCP refused, no version message, handshake dropped, wrong protocol (not BSV), connection error.
4. Publishes verified-alive peers to the DNS seed at `seed.indelible.one`.

The first crawler run revealed a critical insight about the BSV network:

```
26 alive | no_version: 934, refused: 149, no_verack: 149,
not_bsv: 17, dropped: 10, error: 2
```

74% of "known" peers were ghosts — IP addresses that used to run BSV nodes years ago and now run something else entirely, or nothing at all. The peer databases inherited from legacy DNS seeds were polluted with dead entries. The crawler is what prunes the ghosts: it distinguishes live soil from dead soil, giving the network its first accurate picture of its own topology.

Other bridges fetch the crawler's verified peer list on startup, supplementing their own discovery mechanisms. This provides a federation-internal bootstrap path that operates independently of all legacy BSV DNS seeds.

5.9 Good Peer Persistence

Peers that send valid block headers are promoted to "good peer" status and saved to `good-peers.json` in the bridge's data directory. On restart, good peers are loaded first and connected before any DNS seed resolution or peer exchange.

Good peer status is not permanent. A peer that fails to connect 3 times consecutively is demoted — removed from the good peers map and returned to the general peer pool with standard cooldown timers. The `consecutiveFails` counter resets to 0 when a peer successfully sends valid headers.

This persistence layer means a bridge restart is not a cold start. The bridge remembers which peers were reliable and reconnects to them first, then discovers new peers through the standard four-stage discovery process (Section 5.1).

6. Header Synchronisation Layer

The header synchronisation layer downloads and stores all BSV block headers, as specified in Section 8 of the Bitcoin whitepaper [1].

6.1 Header Download

Headers are requested using the `getheaders` message with a block locator -- a list of known block hashes at exponentially decreasing heights, always including the genesis block hash. This locator allows the remote peer to find the common point in the chain and send subsequent headers. Each response contains up to 2,000 headers.

6.2 Storage

Headers are stored in LevelDB with dual indexing: by height (`header:{height}`) and by hash (`height:{hash}`). This permits both sequential traversal and random access by block hash, the latter being essential for Merkle proof verification.

At the time of writing, the header chain spans 945,000+ blocks. Initial synchronisation of this chain requires the `NODE_OPTIONS='--max-old-space-size=2048'` environment variable to avoid out-of-memory conditions during the download of approximately 75MB of header data. All production bridges run with 2GB heap allocation — the default 512MB is insufficient for bridges maintaining 15+ peer connections, a full header chain, LevelDB indexes, and in-memory caches simultaneously.

6.3 Merkle Proof Verification

As described in Section 7 of the Bitcoin whitepaper [1], each block header contains a Merkle root that commits to every transaction in the block. The header sync layer provides Merkle proof verification: given a transaction hash, a Merkle branch, a transaction index, and a block hash, the layer computes the Merkle root from the proof and compares it against the stored block header's Merkle root. A match proves the transaction's inclusion in the block with mathematical certainty.

7. Federation Layer

The federation layer replaces the static peer lists and shared-secret authentication of earlier relay mesh designs with an open, decentralised protocol. Any bridge can join by registering on-chain. Trust is earned through observed behaviour, not granted by configuration.

7.1 Purpose

The federation layer handles three problems:

1. **Discovery:** How does a new bridge find existing bridges to peer with?
2. **Identity:** How does a bridge prove it is who it claims to be?
3. **Trust:** How does a bridge decide which peers to keep and which to disconnect?

The solutions are: on-chain registry for discovery, cryptographic handshake for identity, and local peer scoring for trust.

7.2 On-Chain Registry

Bridge registration is published on-chain using CBOR-encoded OP_RETURN transactions. The protocol prefix `indelible.bridge-registry` identifies registry transactions in the OP_RETURN data.

A registration transaction has two outputs:

1. **OP_RETURN output** (0 satoshis): Contains the protocol prefix followed by a CBOR-encoded payload with the following fields:

Field	Type	Description
<code>action</code>	string	"register" or "deregister"

endpoint	string	WebSocket endpoint (e.g., <code>wss://bridge.example.com:8333</code>)
pubkey	Uint8Array (33 bytes)	Compressed public key -- the bridge's identity
capabilities	string[]	Subset of: <code>tx_relay</code> , <code>header_sync</code> , <code>broadcast</code> , <code>address_history</code>
versions	string[]	Supported protocol versions (e.g., <code>["1.0"]</code>)
network_version	string	Current network version
stake_txid	Uint8Array (32 bytes)	Transaction ID of the stake bond
mesh_id	string	Mesh identifier (e.g., <code>"70016"</code>)
timestamp	number	Unix timestamp in seconds

2. **Dust output** (100 satoshis): Sent to the beacon address (`1Khh4VshyN8PnzxbTSjiojcQbbABNSZyzR`), a deterministic address derived from SHA-256 of the protocol prefix. This makes all registry transactions discoverable via a single address history query.

Deregistration uses the same protocol prefix with `action: "deregister"` , the bridge's pubkey, a reason string, and a timestamp. The latest transaction for a given pubkey wins, supporting endpoint updates and re-registration.

7.3 Stake Bonds

Before registering, a bridge must create a stake bond -- a transaction that locks a minimum of 1,000,000 satoshis (~0.01 BSV) to the bridge operator's own address. The stake bond transaction ID is included in the registration payload.

Parameter	Value
Minimum	1,000,000 sats (~0.01 BSV)
Purpose	Proof of BSV ownership, Sybil deterrence
Scoring weight	10% (stake_age factor in peer scoring)
Recovery	Deregister to unlock

The stake is not a payment -- it is a security deposit locked to the operator's own address. The economic cost of registering fake bridges provides Sybil deterrence. Higher stakes do not buy significant advantage in scoring (only 10% weight); the real defense against bad actors is data accuracy scoring (40% weight).

Operators can voluntarily stake more than the minimum for a slightly higher trust score, but the marginal benefit is logarithmic -- doubling the stake adds minimal score improvement.

7.4 Peer Discovery

Bridges discover each other through three mechanisms:

1. **Seed peers**: On startup, the bridge connects to peers listed in its `seedPeers` configuration. These are known bridges with verified pubkeys.
2. **Gossip protocol**: Once connected to at least one peer, the bridge's gossip manager requests peer lists (`getpeers` message) and broadcasts signed announcements (`announce` message). Announcements include the bridge's pubkey, endpoint, mesh ID, and timestamp, signed with the bridge's private key. Peers verify the signature and propagate valid announcements to their own peers, creating a flooding discovery mechanism.
3. **On-chain beacon watcher**: The bridge monitors the beacon address for new registration transactions. When a new registration arrives on-chain, the bridge extracts the CBOR payload, verifies the registration fields, and adds the peer to its

known peer set.

The gossip protocol provides real-time discovery (seconds), while the on-chain registry provides permanent, auditable discovery (minutes to hours). Both mechanisms filter by `mesh_id` -- bridges only peer within the same mesh.

7.5 Cryptographic Handshake

Every bridge-to-bridge connection begins with a mutual authentication handshake using BSV keypairs. The protocol requires three messages:

1. Initiator → Responder: { type: "hello", pubkey, nonce, versions, endpoint }
2. Responder → Initiator: { type: "challenge_response", pubkey, nonce, signature, selected_version }
3. Initiator → Responder: { type: "verify", signature }

Step 1: The initiator sends a hello containing its compressed public key, a 32-byte random nonce, its supported protocol versions, and its endpoint.

Step 2: The responder verifies the initiator's pubkey against the set of registered pubkeys (if available). It selects the highest mutually supported protocol version. It signs the initiator's nonce with its own private key (proving identity) and sends a challenge_response containing its own pubkey, a new nonce, the signature, and the selected version.

Step 3: The initiator verifies the responder's signature against the responder's pubkey and the original nonce. If valid, it signs the responder's nonce and sends a verify message.

The responder verifies the final signature. If valid, the connection is established with mutual authentication complete. Both sides have proven possession of the private keys corresponding to their claimed public keys.

Connections that do not complete the handshake within 10 seconds are dropped.

7.6 Real-Time Transaction Relay

Once peered, bridges relay transactions to each other in real time over persistent WebSocket connections. When a transaction is broadcast through one bridge (via its API or P2P layer), the bridge propagates it to all connected peer bridges immediately. This provides push-based propagation -- peer bridges receive the transaction without waiting for the Bitcoin P2P network to propagate it naturally.

7.7 Header Sharing

Bridges share block headers with peers as they arrive. When a bridge receives a new header from the Bitcoin P2P network, it propagates the header to its peer bridges. This ensures that all bridges in the mesh maintain synchronised header chains, even if individual bridges have intermittent Bitcoin P2P connectivity.

8. API Layer

8.1 REST API

The API layer exposes HTTP endpoints for bridge operators and client applications:

- `GET /status` -- Bridge status: pubkey, mesh ID, uptime, peers, headers, mempool
- `GET /tx/:txid` -- Retrieve a transaction by identifier with full protocol parsing (see Section 12)
- `GET /tx/:txid/status` -- Transaction lifecycle state (mempool, confirmed, orphaned, dropped)
- `GET /proof/:txid` -- Merkle proof for confirmed transactions with block context
- `GET /mempool` -- Current mempool with parsed outputs and protocol badges
- `GET /inscriptions` -- Query indexed inscriptions by mime type, address, or time range
- `GET /inscription/:txid:vout/content` -- Serve raw inscription content with CAS resolution
- `GET /price` -- Live BSV/USD exchange rate with 60-second cache

- `GET /tokens` -- List all deployed BSV-20 tokens indexed by the bridge
- `GET /token/:tick` -- Deploy info for a specific BSV-20 token
- `GET /token/:tick/balance/:scriptHash` -- Token balance for an owner by script hash
- `GET /apps` -- Health status of applications running on the bridge
- `GET /discover` -- List all bridges in the mesh with their endpoints and capabilities
- `GET /api/address/:addr/balance` -- Confirmed and unconfirmed balance for an address
- `GET /api/address/:addr/history` -- Full transaction history for an address
- `GET /api/address/:addr/unspent` -- Unspent transaction outputs for an address
- `GET /api/tx/:txid/hex` -- Raw transaction hex
- `POST /broadcast` -- Broadcast a raw transaction to the Bitcoin network
- `POST /api/sessions/index` -- Index Indelible session metadata
- `GET /api/sessions/:address` -- List sessions for an address

8.2 Operator Dashboard

Each bridge runs a local HTTP server (default port 9333) with a glassmorphism dashboard featuring a 3D mesh topology map powered by Three.js. The dashboard provides six tabs:

- **Overview:** Bridge stats, peer connections, header height, wallet balance, 3D network topology
- **Mempool:** Real-time transaction list with protocol badges and decoded data
- **Tx Explorer:** Transaction lookup with full protocol parsing
- **Inscriptions:** Browser for on-chain inscriptions with mime type and address filtering
- **Tokens:** BSV-20 token list, deploy info, balance queries
- **Apps:** Health checks, SSL status, and latency monitoring for applications running on the bridge

Operator access is authenticated via a per-bridge `statusSecret` generated during initialisation.

8.3 Security

The API layer implements layered security:

- **Operator authentication:** Dashboard access requires the bridge's `statusSecret`.
- **Bridge-to-bridge authentication:** Peer bridges authenticate via the cryptographic handshake (Section 7.5), not API keys. A bridge proves its identity by signing nonces with its BSV private key.
- **Rate limiting:** Unauthenticated requests are rate-limited.
- **CORS:** Configurable origin whitelisting for browser-based access.

8.4 Application Integration Model

Applications built on the relay mesh are **consumers** of the federation REST API, not tenants hosted on bridge infrastructure. A bridge is a piece of infrastructure — like a database or CDN node — that applications connect to over the network. The application itself runs wherever its developer chooses: a cloud platform, a dedicated server, a static hosting service, or a local machine.

This separation follows naturally from the API layer design. Section 8.1 defines the HTTP endpoints that bridges expose. Any HTTP client — a web application backend, a CLI tool, a browser — can call these endpoints. The bridge does not need to know what application is calling it, and the application does not need to run on the same machine as the bridge.

The recommended integration pattern uses multiple bridges for resilience. An application maintains a list of known bridge endpoints (seeded manually or via the `/discover` endpoint) and round-robins requests across them with per-request timeouts. If a bridge is unreachable, the application tries the next. This mirrors how the bridges themselves handle peer failures in the mesh layer.

The SDK (`@relay-federation/sdk`) abstracts bridge selection for applications that prefer a library over raw HTTP. For applications that need full control — custom retry logic, request signing, or integration with existing HTTP infrastructure — the REST API is the primary interface.

The Apps tab in the operator dashboard (Section 8.2) serves a monitoring function: bridge operators can configure health checks for applications that depend on their bridge, regardless of where those applications are hosted. This is observational, not operational —

the bridge monitors the application, it does not host or manage it.

See [Building Apps on the Federation](#) for implementation details, code examples, and a production checklist.

9. Supervision & Self-Healing

9.1 Purpose

The Bitcoin whitepaper addresses crash tolerance directly. Nakamoto states in the Abstract [1]:

"Nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone."

Section 5 [1] elaborates:

"Block broadcasts are also tolerant of dropped messages. If a node does not receive a block, it will request it when it receives the next block and realizes it missed one."

The supervision layer implements this principle for the relay mesh.

9.2 The Promise Double-Reject Problem

During a production audit, multiple bridges crashed simultaneously. The root cause was a class of bug present across multiple layers: the promise double-reject.

In Node.js, a Promise executor that calls `reject()` more than once produces an unhandled rejection on the second call. The first rejection is caught by the `.catch()` handler. The second, having no handler, propagates as an `unhandledRejection` event. If this event has no listener, Node.js terminates the process.

The bug pattern was identical across all affected layers. Network operations used Promises with both a timeout handler and an error/close handler. When a connection timed out and then closed (or closed and then timed out), both handlers called `reject()`. The fix uses a settled flag guard:

```
let settled = false;
const timeout = setTimeout(() => {
  if (settled) return;
  settled = true;
  reject(new Error('timeout'));
}, 10000);
socket.on('close', () => {
  if (settled) return;
  settled = true;
  reject(new Error('closed'));
});
```

A process-level `unhandledRejection` handler provides a safety net for any unforeseen double-rejects, logging the error and preventing process termination.

9.3 Recovery After Restart

When a bridge restarts, it re-establishes all layers:

1. **Good peer warm start:** Loads `good-peers.json` (Section 5.9) and connects to peers that proved reliable in the previous session.
2. **P2P reconnection:** Re-establishes TCP connections to BSV full nodes via the four-stage discovery process (Section 5.1).
3. **Header chain re-synchronisation:** Requests headers from its last known height using `getheaders` with a block locator.

4. **Federation re-peering:** Reconnects to seed peers, resumes gossip announcements, and re-establishes authenticated WebSocket connections.
5. **Beacon backfill:** Scans the on-chain beacon address for any bridges that registered while this bridge was offline.
6. **Peer scoring reset:** Peers receive a default stake age on reconnection; scoring resumes with fresh uptime and latency measurements.

9.4 Process Supervision with systemd

All production bridges are managed by `systemd` — the Linux process supervisor. The bridge registers as a service with `StandardOutput=journal` and `StandardError=journal`, ensuring all logs are captured in the systemd journal for debugging and audit. If the bridge process crashes, `systemd` restarts it automatically, and the recovery sequence described above rebuilds all layers.

This makes the "rejoin the network at will" principle (Section 1) fully automatic. A bridge crash is a temporary absence, not a permanent failure — the network heals around the gap and the bridge resumes when the process restarts.

10. Transaction Lifecycle

10.1 Client Initiation

A client application -- whether a web application, CLI tool, or API integration -- constructs a transaction, signs it using `@bsv/sdk`, and submits the raw transaction hex to a bridge's `/broadcast` endpoint.

10.2 Broadcast

The API layer passes the raw transaction hex to the SPV client's broadcast method. The SPV client computes the transaction identifier, stores the raw transaction, sends an `inv` message to all connected Bitcoin full nodes, and serves the full `tx` message when peers respond with `getdata`.

The federation layer simultaneously relays the transaction to all connected peer bridges via WebSocket, ensuring mesh-wide propagation.

10.3 Client Lookup

When a client requests a transaction, the bridge follows a deterministic lookup chain:

1. **Local store:** LevelDB lookup by txid.
2. **P2P getdata:** A `getdata MSG_TX` request to connected Bitcoin full nodes. `notfound` responses are handled immediately.
3. **Peer bridges:** WebSocket relay from peer bridges that may have the transaction.
4. **404:** If no source produces the transaction.

10.4 Confirmation

The bridge's periodic header synchronisation detects new blocks. As described in Section 7 [1], the Merkle tree structure allows the bridge to verify that a transaction is included in a block by checking a Merkle branch against the block header's Merkle root.

Every transaction the bridge sees is tracked through a lifecycle state machine with four states: `mempool` (seen but unconfirmed), `confirmed` (proven in best chain with Merkle proof), `orphaned` (was confirmed but block disconnected by reorg), and `dropped` (mempool expiry after 14 days). The `txStatus` sublevel stores the authoritative state; the `txBlock` sublevel stores block placement with a reverse index (`block!<blockHash>!tx!<txid>`) that enables efficient rollback during chain reorganisations.

When a reorg occurs, the bridge identifies disconnected blocks via the reverse index, marks all affected transactions as orphaned, and re-enqueues them for confirmation against the new best chain. All rollback operations execute in a single atomic LevelDB `batch()` write to prevent inconsistent state.

Clients query confirmation state via `GET /tx/:txid/status` and retrieve Merkle proofs via `GET /proof/:txid`.

11. Security Considerations

11.1 Authentication and Access Control

The system employs layered authentication:

- **Operator secret:** Per-bridge authentication for dashboard access.
- **Cryptographic handshake:** Bridge-to-bridge authentication via mutual nonce signing (Section 7.5).
- **Rate limiting:** Per-IP throttling for unauthenticated requests.

11.2 Data Encryption

Applications using the relay mesh can encrypt payloads before broadcast. The relay mesh itself is transport-agnostic -- it relays transactions without inspecting their content.

11.3 Loop Prevention

Gossip announcements include a deduplication mechanism: each announcement is identified by `pubkeyHex:timestamp`, and seen announcements are stored in a dedup set. Bridges do not re-broadcast announcements they have already seen, preventing infinite propagation loops.

11.4 The Bloom Filter Ban

As noted in Section 2.3, sending a `filterload` message to a BSV full node results in an immediate Misbehaving score of 100. The system avoids this entirely by never using bloom filters. All transaction lookups use direct `getdata MSG_TX` with explicit transaction identifiers.

11.5 Protocol Compliance — The 10 Rules

The following table documents every protocol behaviour that BSV full nodes enforce on peers, derived from reading the BSV node source code (C++, `bitcoin-sv-1.1.1`) alongside the bridge code (JavaScript). Violating any rule results in disconnection, deprioritisation, or banning. The federation implements all ten.

#	BSV Node Requirement	Enforcement Mechanism	Federation Implementation
1	<code>inv/getdata/tx</code> three-step handshake (Section 2.2)	Unsolicited tx messages are silently ignored by <code>ProcessMessage()</code> in <code>net_processing.cpp</code> . The transaction never enters the mempool.	<code>bsv-node-client.js</code> — broadcasts always send <code>inv</code> first, serve tx only on <code>getdata</code> response.
2	<code>services=0n</code> — do not claim <code>NODE_NETWORK</code> (Section 4.1.1)	Claiming <code>services=1n</code> (<code>NODE_NETWORK</code>) causes full nodes to send <code>getdata MSG_BLOCK</code> . Inability to serve blocks triggers misbehaving penalties.	<code>bsv-peer.js</code> — version message sets <code>services=0n</code> (<code>NODE_NONE</code>).
3	No bloom filters (Section 2.3)	<code>filterload</code> triggers <code>Misbehaving(peer, 100)</code> in <code>ProcessMessage()</code> → immediate 24-hour ban. <code>DEFAULT_PEERBLOOMFILTERS=false</code> on BSV.	Never sent. All tx lookups use direct <code>getdata MSG_TX</code> with explicit txids.
4	Current block height in version message (Section 5.6, Fix 1)	Full nodes compare the peer's advertised height against their own chain tip. Stale height → deprioritised as inactive node.	Headers sync to chain tip before connecting to remaining peers. Real height advertised.
5	Novel tx relay — first relayer wins (Section	<code>nLastTxTime</code> in <code>CNodeState</code> only updates when a peer delivers a tx the node has never seen. Layer 3 of	Immediate <code>inv</code> forwarding, shared <code>_txCache</code> , <code>mesh→P2P</code>

	5.4, 5.6)	eviction protects the 4 peers with most recent <code>nLastTxTime</code> . No novel relay = zero eviction protection.	relay. 52-69% hit rates.
6	Don't reconnect aggressively (Section 5.6, Fix 4)	<code>GetChance()</code> in <code>addrman.cpp</code> applies 99% penalty for retries within 10 minutes. <code>IsTerrible()</code> marks peers "terrible" after 3 failures. Exponential 0.66x backoff per failure.	120-second cooldown, concurrency guard, max 20 connections/cycle, 3-fail demotion from good peers.
7	Respond to pings	Full nodes send <code>ping</code> with a 64-bit nonce. Missing <code>pong</code> response → connection scored as unresponsive and eligible for eviction.	<code>bsv-peer.js</code> — automatic <code>pong</code> with matching nonce on every <code>ping</code> received.
8	Send <code>protoconf</code> after handshake (Section 4.2)	Protocol version 70016+ requires <code>protoconf</code> declaring maximum receive payload size. Missing <code>protoconf</code> may cause failures on large messages.	Sent immediately after <code>verack</code> — advertises 2MB max receive payload.
9	Handle <code>authch</code> silently (Section 4.2)	Mining nodes send <code>authch</code> (authentication challenge) for <code>MinerID</code> . Crashing or disconnecting on unrecognised messages marks the peer as unreliable.	Logged and ignored. Non-mining SPV clients have no <code>MinerID</code> key. Connection proceeds normally.
10	<code>relay=0</code> in version message	SPV clients that set <code>relay=1</code> may receive bloom-filtered data or be expected to relay blocks. Incorrect relay flag creates protocol expectation mismatch.	Version message sets <code>relay=0</code> . Selective relay through explicit <code>inv</code> forwarding, not the bloom filter path.

These rules are not suggestions — they are enforced by the BSV node source code. The bridge implements all ten, which is why peer connections persist for hours instead of minutes.

11.6 Stake Bond Anti-Sybil

Registering a bridge requires locking a minimum of 1,000,000 satoshis in a stake bond. This economic cost prevents trivial mass registration of fake bridges. The stake bond transaction ID is included in the on-chain registration payload, making it publicly verifiable.

Stake age is factored into peer scoring (Section 11.7): longer-held stakes contribute to a higher trust score. This creates an ongoing cost for maintaining fake identities -- the attacker's capital remains locked for the duration.

11.7 Peer Scoring

Every bridge scores its peers locally. There is no centralised reputation authority.

```
score = 0.3 * uptime + 0.2 * response_time + 0.4 * data_accuracy + 0.1 * stake_age
```

Factor	Weight	What It Measures	Normalisation
Uptime	0.3	Percentage reachable over rolling window	pongs / pings (1000-sample window)
Response time	0.2	Normalised inverse latency	1.0 at ≤ 100ms, 0.0 at ≥ 5000ms, linear between
Data accuracy	0.4	Percentage of relayed data that validates correctly	good / total (1000-sample window)
Stake age	0.1	How long the stake bond has existed	$\log_2(\text{days}) / 10$, capped at 1.0

Data accuracy is weighted highest (40%) because correct data is the primary function of a relay bridge. A bridge that relays invalid headers or transactions is worse than useless -- it wastes bandwidth and can mislead clients.

Score thresholds:

- Score < 0.3 → auto-disconnect
- Score < 0.1 → 24-hour blacklist

Each bridge computes scores independently. No consensus is required. A bridge that behaves well from one peer's perspective but poorly from another's will have different scores on different bridges. This is by design -- local scoring reflects local observations.

11.8 Pubkey Authentication

Bridge identity is tied to a BSV keypair generated during initialisation. The compressed public key is the bridge's permanent identity. The on-chain registration proves key ownership (the registration transaction is signed by the bridge's private key). The cryptographic handshake (Section 7.5) proves identity on every connection.

This prevents endpoint spoofing: an attacker cannot claim to be a registered bridge without possessing the corresponding private key.

11.9 Gossip Signature Verification

All gossip announcements are signed by the announcing bridge's private key. The signature covers the pubkey, endpoint, mesh ID, and timestamp. Receiving bridges verify the signature before accepting or propagating the announcement. This prevents impersonation -- a bridge cannot announce itself as another bridge without possessing that bridge's private key.

Announcements older than 5 minutes or more than 30 seconds in the future are discarded, preventing replay attacks.

12. Protocol Parsing

The relay mesh parses BSV transactions beyond simple P2PKH. Every output is typed, protocol-detected, and returned as structured data via the API and dashboard.

12.1 Supported Output Types

Type	Detection	Parsed Fields
p2pkh	76a914{20 bytes}88ac	hash160
op_return	6a... Or 006a...	data[], protocol, parsed
ordinal	Contains 0063036f7264	contentType, content, isBsv20, bsv20
p2sh	a914{20 bytes}87	scriptHash
multisig	Ends with ae	m, n, pubkeys[]

12.2 Supported Protocols (inside OP_RETURN)

Protocol	Prefix Address	Parsed Fields
B://	19HxigV4QyBv3tHpQVcUEQyq1pzZVdoAut	data, mimeType, encoding, filename
BCAT	15DHFxwZJT58f9nhyGnsRBqrgwK4W6h4Up	info, mimeType, charset, filename, chunkTxids[]
BCAT-part	1ChDHzzd1H4wSjgGMHyndZm6qxEDGjqpJL	data
MAP	1PuQa7K62MiKctssSLKy1kh56WWU7MtUR5	action, pairs: { key: value }

MetaNet	Magic bytes 6d657461	nodeAddress, parentTxid
BSV-20	Ordinal with application/bsv-20	bsv20: { p, op, tick/id, amt, ... }

This parsing enables the dashboard to display protocol badges and decoded data for every transaction, transforming raw hex into human-readable information.

Additionally, every parsed output now includes a `scriptHash` field -- the SHA256 hash of the output's locking script hex. This provides a universal owner identifier that works for P2PKH, P2PK, P2SH, bare scripts, and any locking script type, unlike address-based keying which only works for standard P2PKH outputs.

13. Indexing Layer

The indexing layer transforms the bridge from a stateless relay into a self-verifying data source. It stores transaction confirmation state, Merkle proofs, BSV-20 token balances, and inscription content -- all anchored to the header chain for reorg safety.

13.1 Transaction Confirmation Tracking

Every transaction seen by the bridge is tracked through a lifecycle state machine (Section 10.4). The `txStatus` sublevel stores the authoritative state for each transaction. The `txBlock` sublevel stores block placement with Merkle proofs and a reverse index for efficient reorg rollback.

13.2 Content-Addressed Inscription Storage

Inscription content is stored in a content-addressed system keyed by SHA256 hash. Small payloads (< 4 KB) are stored inline in LevelDB; larger payloads are written to the filesystem at `data/content/<first2chars>/<hash>`. The inscription record stores only metadata (content hash, length, MIME type, location pointer), reducing LevelDB compaction pressure and enabling deduplication when the same content appears in multiple transactions.

Content is served via `GET /inscription/:txid/:vout/content` with immutable cache headers (`Cache-Control: public, max-age=31536000, immutable`). The serving path resolves content through the CAS layer, falling back to re-extraction from the raw transaction if the CAS entry is missing.

13.3 BSV-20 Token Indexing

The bridge indexes BSV-20 deploy and mint operations. Token indexing operates on **confirmed transactions only** -- mempool token operations are excluded to prevent double-spend corruption of balances.

Token owners are identified by script hash (SHA256 of locking script hex) rather than address, supporting all output types. The state machine enforces BSV-20 rules: first deploy wins (chain-ordered by block height), tick normalisation to lowercase, per-mint amount limits, and supply cap enforcement via BigInt arithmetic.

Every token operation is applied as a single atomic LevelDB `batch()` write: token record update, balance credit, operation log entry, and idempotency marker. Operation keys use zero-padded heights for lexicographic ordering, ensuring deterministic replay. Idempotency markers (`applied!<txid>`) prevent duplicate processing across restarts and backfill reruns.

13.4 Historical Backfill

The `relay-bridge backfill` CLI command walks historical blocks from a configured start height to the chain tip. For each block, it fetches the transaction list from WhatsOnChain (one API call per block), then selectively fetches raw transaction data only for transactions matching interest filters (ordinal inscriptions, BSV-20 operations). This avoids fetching entire blocks while still indexing relevant historical data.

Backfill supports resume via a stored `meta.backfill_height` value. Rate limiting (350ms between API calls) prevents upstream throttling. Progress is logged every 100 blocks.

13.5 Price Feed

The bridge provides a live BSV/USD exchange rate via `GET /price`, cached in memory with a 60-second TTL. The price is sourced from WhatsOnChain's exchange rate endpoint. This enables applications to display fiat-denominated values without maintaining their own price feed infrastructure.

14. Data Relay Layer

The preceding layers handle transactions and block headers -- data that originates on the Bitcoin network and has permanent on-chain representation. Applications built on the relay mesh, however, frequently need to distribute information that is not a transaction and has no UTXO. Exchange rate quotes, service attestations, availability announcements, and operational signals are ephemeral by nature: valuable for seconds or minutes, meaningless thereafter, and wasteful to record on-chain.

The existing BSV protocol stack does not address this requirement. BRC-22 overlay synchronisation [4] operates on UTXOs admitted to topic-specific databases -- it requires on-chain transactions as input. BRC-33 PeerServ [5] provides point-to-point addressed message delivery -- it requires knowing the recipient's identity key. Neither mechanism supports broadcasting short-lived signals to all interested peers via gossip.

The data relay layer fills this gap by extending the bridge wire protocol with ephemeral signed data envelopes.

14.1 Design Principles

The data relay layer is governed by five invariants. If any invariant is violated, the system degenerates into a partial duplicate of existing infrastructure:

1. **Ephemeral.** Envelopes expire after a time-to-live interval and are forgotten. The relay mesh is not a database.
2. **Broadcast.** Envelopes propagate to all interested peers via gossip flood. There is no addressed recipient.
3. **Topic-filtered.** Peers declare interest prefixes. Bridges forward only matching envelopes. Uninterested peers are not burdened.
4. **Off-chain.** Envelopes are not transactions. They carry no UTXO, require no mining fee, and receive no on-chain confirmation.
5. **Payload-opaque.** Bridges verify envelope signatures and enforce size and TTL constraints. They do not interpret payload content. Schema validation, trust decisions, and aggregation logic belong to the application layer.

These invariants establish the boundary between the data relay layer and the rest of the BSV Mandala architecture. Persistent state belongs to overlay services (BRC-22). Transaction submission belongs to ARC. Merkle proofs belong to the Teranode asset server. The relay mesh carries what those systems do not: ephemeral signals that applications need in real time without on-chain overhead.

14.2 Wire Protocol

Four message types are added to the existing WebSocket protocol. The existing twelve message types (hello, challenge_response, verify, getpeers, peers, announce, header_announce, header_request, headers, tx_announce, tx_request, tx) are unchanged.

Signed Data Envelope

```
{
  type:      "data",
  topic:     string,      // hierarchical namespace (e.g. "oracle:rates:bsv")
  payload:   string,      // opaque content -- bridge does not interpret
  pubkeyHex: string,      // originator's compressed secp256k1 public key
  timestamp: number,      // Unix seconds
  ttl:       number,      // seconds until expiry (max 3600)
  signature: string       // ECDSA-SHA256 over (topic + payload + timestamp + ttl)
}
```

Upon receiving a data envelope, the bridge executes the following validation sequence:

1. Verify the ECDSA signature against the claimed public key.
2. Reject if the timestamp is more than 30 seconds in the future.
3. Reject if the envelope has expired (timestamp + TTL < current time).
4. Reject if the payload exceeds 4,096 bytes.
5. Reject if the TTL exceeds 3,600 seconds.
6. Compute a SHA-256 deduplication hash over (pubkeyHex, topic, payload, timestamp). Reject if the hash has been seen before.
7. Store the envelope in a bounded per-topic ring buffer.
8. Forward the envelope to all connected peers whose declared topic interests match, excluding the source peer.

This validation sequence mirrors the discipline of the existing transaction relay: verify first, store second, propagate third. The deduplication mechanism prevents infinite gossip loops, analogous to the `seen` set in transaction relay.

Topic Interest Declaration

```
{
  type:      "topics",
  interests: string[],    // topic prefixes (e.g. ["oracle:", "attestation:"])
  pubkeyHex: string,
  timestamp: number,
  signature: string      // ECDSA-SHA256 over (interests.join(',') + timestamp)
}
```

A bridge declares the topic prefixes it wishes to receive. Matching is by string prefix: an interest of `"oracle:"` matches topics `"oracle:rates:bsv"`, `"oracle:rates:eth"`, and any other topic beginning with that prefix. The wildcard `"**"` matches all topics.

Bridges that declare no interests receive no data envelopes. This is silent by default -- existing transaction relay and header synchronisation continue unaffected. The topic interest system operates on a separate plane from the transaction relay system; the two do not interact.

Data Request and Response

```
Request: { type: "data_request", topic, since, limit }
Response: { type: "data_response", topic, envelopes[], hasMore }
```

A bridge that comes online after a signal was published has missed it. The data request message allows it to query a peer's local ring buffer for cached envelopes newer than a given timestamp. The responding bridge returns matching envelopes up to the requested limit.

The critical design constraint is that data requests are local queries, not mesh-wide searches. A bridge queries a specific peer's cache. The request is not forwarded. The response, when ingested, is stored locally but not re-propagated as gossip. This prevents catch-up operations from generating secondary gossip storms.

14.3 Storage Model

Envelopes are stored in bounded in-memory ring buffers, one per topic. When a buffer reaches its capacity (default: 100 envelopes), the oldest envelope is evicted. Expired envelopes are pruned lazily on read. No envelope is written to disk.

The deduplication set is also bounded and in-memory. When the set reaches its capacity (default: 10,000 entries), the oldest entry is evicted in FIFO order. This prevents unbounded memory growth on long-running bridges while retaining deduplication effectiveness for the recent window.

This storage model is intentionally volatile. A bridge restart clears all cached envelopes. The pull-based catch-up mechanism (`data_request`) provides the recovery path: a restarted bridge queries its peers for recent data on topics of interest.

14.4 HTTP Interface

The data relay layer exposes three HTTP endpoints on the bridge status server, extending the existing REST API:

Endpoint	Method	Purpose
<code>/data</code>	POST	Submit a signed envelope for injection into the gossip mesh
<code>/data/topics</code>	GET	List topics with cached data, including envelope count and latest timestamp per topic
<code>/data/:topic</code>	GET	Query cached envelopes by topic, with optional <code>since</code> timestamp and <code>limit</code> pagination

These endpoints are the application-facing surface. An application never speaks the wire protocol directly -- it submits and queries envelopes via HTTP, and the bridge handles gossip propagation internally. This mirrors the existing pattern: applications broadcast transactions via `POST /broadcast` and the bridge handles `inv / getdata / tx` propagation to peers.

Payment for data operations via BRC-105 HTTP micropayments [6] is architecturally planned but not yet implemented. The insertion points are identified in the endpoint handler code. When activated, bridges will be able to charge per-envelope for propagation and per-query for retrieval, using the same 402-based payment flow that the BSV ecosystem has standardised for HTTP service monetisation.

14.5 Application Examples

The data relay layer is deliberately minimal. Its value comes from the diversity of applications it enables without protocol changes:

Real-time rate oracle. An exchange rate service publishes `{topic: "oracle:rates:bsv", payload: '{"USD":42.50}', ttl: 60}` every 30 seconds. Every bridge with a declared interest in `"oracle:"` receives the quote within seconds. Applications poll their local bridge's cache or query on demand. Multiple independent oracles can publish to the same topic; the application decides which sources to trust and how to aggregate conflicting values.

Attestation relay. A purchase order approval system publishes `{topic: "attestation:svcp.po.v1:instanceId", payload: '{"evidenceHash":"abc...}', ttl: 300}`. Interested parties receive the attestation in real time without waiting for on-chain confirmation of the underlying evidence transaction.

Service availability. An overlay node publishes `{topic: "overlay:status:tokens", payload: '{"load":0.7,"maintenance":false}', ttl: 120}` to signal its current operational state. Applications route traffic accordingly.

In each case, the bridge carries opaque bytes. The topic namespace, payload schema, trust model, and aggregation logic are entirely application concerns. The relay mesh provides the transport; the application provides the meaning.

14.6 Scope Boundary

The data relay layer is a complement to existing BSV infrastructure, not a replacement. The boundary is defined by what the layer explicitly does not do:

- It does not persist data beyond TTL. Persistent state belongs to overlay services.
- It does not submit transactions. Transaction broadcast belongs to ARC and the existing transaction relay.
- It does not serve Merkle proofs. Proof authority belongs to the Teranode asset server.
- It does not maintain a topic registry. Topics are conventions, not registered infrastructure.
- It does not filter transactions by address or script. Transaction-level filtering is a separate concern with different design constraints.

This boundary ensures that the data relay layer remains lightweight, focused, and complementary to the broader BSV architecture as it evolves toward Teranode-scale operation.

15. x402 Payment Middleware

The relay mesh provides free read access to all API endpoints by default. Bridge operators can optionally enable HTTP 402-based micropayments to earn satoshis from write operations, creating an economic incentive to run infrastructure.

15.1 Design: Free Reads, Paid Writes

The payment model follows a simple principle: reading data costs nothing, writing data costs satoshis. Transaction lookups, UTXO queries, balance checks, header verification, and mempool inspection are free. Transaction broadcast, session indexing, and data envelope injection can be priced per-endpoint.

This asymmetry is deliberate. Free reads attract application developers — the barrier to building on the federation is zero. Paid writes generate revenue for bridge operators — the incentive to run bridges grows with usage. The two-sided flywheel: more apps create more write traffic, more write revenue incentivises more bridges, more bridges improve redundancy and latency for apps.

15.2 Protocol

When a client sends a request to a paid endpoint without payment, the bridge responds with HTTP 402 (Payment Required) and a JSON body specifying the price in satoshis and the bridge operator's BSV address. The client constructs a BSV transaction paying the specified amount, broadcasts it, and retries the original request with the transaction ID in the `X-Payment-TxId` header.

The bridge verifies the payment transaction exists in the mempool (checking both its own mempool and the P2P network), confirms the output pays the correct address for the correct amount, and processes the original request.

15.3 Replay Protection

Each payment transaction ID is recorded in a single-key LevelDB entry (`u!{txid}`). A transaction ID that has already been used is rejected. This prevents replay attacks where a single payment is reused for multiple requests.

15.4 Configuration

```
{
  "x402": {
    "enabled": true,
    "payTo": "10operatorAddress...",
    "endpoints": {
      "POST /api/broadcast": 1000
    }
  }
}
```

Operators set prices per-endpoint in satoshis. Endpoints not listed in the `endpoints` map remain free. The entire x402 layer is disabled by default — a bridge with no `x402` configuration serves all endpoints for free.

15.5 Compatibility

The implementation follows the [x402 protocol](#) specification used by Coinbase and Cloudflare on Ethereum/Base/Solana, adapted for BSV. The same HTTP 402 flow applies — the difference is that payment is verified on-chain via BSV transaction rather than an EVM smart contract. This positions federation bridges as participants in the broader x402 ecosystem, which has processed 75M+ transactions across multiple chains.

15.6 Operator Dashboard

The bridge dashboard includes an x402 tab showing:

- Total revenue earned (satoshis and USD equivalent)
- Per-endpoint pricing table
- Payment receipt log with transaction IDs
- Operator configuration controls

16. Performance

16.1 Production Deployment

The deployment comprises seven federation bridge nodes across six geographic locations:

Node	Location	RAM	BSV Peers	Novel Relay	Uptime
bridge-alpha	Dallas, TX	2GB	12-18	52% hit rate	systemd managed
bridge-beta	New Jersey	2GB	17-20	69% hit rate	systemd managed
bridge-gamma	Chicago, IL	2GB	4-6	52% hit rate	systemd managed
bridge-delta	Dallas, TX	2GB	3-20	moderate	systemd managed
bridge-epsilon	Atlanta, GA	2GB	4-20	moderate	systemd managed
bridge-6	Silicon Valley	2GB	3-5	active	systemd managed
bridge-7	Silicon Valley	2GB	4-7	15% (crawler node)	systemd managed

All bridges are registered on-chain on mesh `70016` with 1,000,000-satoshi stake bonds. All run with `NODE_OPTIONS='--max-old-space-size=2048'` to prevent out-of-memory crashes during extended operation — the default 512MB Node.js heap is insufficient for bridges maintaining 15+ peer connections, a full header chain, and LevelDB indexes simultaneously. All are managed by `systemd` with journal logging for crash recovery and audit.

Bridge-7 additionally runs the DNS seed crawler (Section 5.8), probing 2,100+ known peers and publishing verified-alive peers to `seed.indelible.one`.

16.2 Broadcast Performance

Transaction broadcasts reach 15-20 Bitcoin full nodes per bridge via the P2P layer. With novel transaction relay (Section 5.6), bridges achieve 52-69% hit rates — meaning over half of relayed transactions are novel to the receiving BSV node. Inter-bridge relay via WebSocket adds sub-second mesh-wide propagation, and mesh-to-P2P relay ensures federation transactions reach the BSV network through all seven bridges simultaneously.

16.3 Header Synchronisation

Initial synchronisation of the full 945,000+ block header chain takes approximately 15-30 minutes. Subsequent synchronisation completes in under one second per new block. The `--max-old-space-size=2048` option is required during initial sync to accommodate the ~75MB header dataset in memory.

16.4 Federation Overhead

Operation	Cost
CBOR registration tx	~300 bytes OP_RETURN + 100 sats dust
Stake bond	1,000,000 sats (locked to operator's own address)
Cryptographic handshake	3 messages, <100ms

Gossip announcement	~200 bytes, every 60 seconds
Peer scoring	Computed locally, zero network cost

17. Related Work

17.1 The Bitcoin Whitepaper

This system is a direct implementation of the principles described by Nakamoto [1]. Section 8 of the whitepaper provides the theoretical foundation for SPV. Section 5 defines the broadcast protocol. Section 7 describes the Merkle tree structure. Section 2 establishes the transaction model. The whitepaper establishes crash tolerance: "Nodes can leave and rejoin the network at will."

17.2 Overlay Network Architectures

Wright [3] describes overlay networks built atop the Bitcoin peer-to-peer layer, with attention to structured routing (Pastry DHT), reputation scoring, and eclipse attack resistance. The federation layer described in this paper implements several of the mechanisms Wright described: reputation scoring (peer scorer), node admission (on-chain registry with stake bonds), and cryptographic identity verification (handshake protocol).

17.3 Electrum and Similar SPV Systems

Electrum-style SPV systems use a client-server model where dedicated indexing servers serve SPV clients over a custom protocol. While effective, this model introduces a trusted third party. The relay mesh eliminates this dependency -- bridges connect directly to Bitcoin full nodes and maintain their own header chains.

17.4 Centralised Node Providers

Ethereum has Infura and Alchemy. Solana has Helius and QuickNode. These services provide API access to blockchain networks without running a full node. They solve the same problem the relay mesh solves -- but through centralisation. One company controls access, pricing, and uptime. The relay mesh achieves the same convenience with decentralised infrastructure. Every developer runs their own bridge.

18. Future Work

18.1 Merkle Tree Pruning

Section 7 of the Bitcoin whitepaper [1] describes reclaiming disk space through Merkle tree pruning. Implementing this would allow long-running bridges to discard old transaction data while retaining verification capability.

18.2 Advanced UTXO Management

Pre-splitting UTXOs into parallel chains would enable concurrent broadcast operations without contention, relevant for high-throughput applications.

18.3 Pastry DHT Routing

Replacing the current gossip-based discovery with DHT-based routing would reduce the $O(n)$ message cost of announcement flooding to $O(\log n)$, enabling the mesh to scale to hundreds of bridges.

18.4 Cross-Mesh Isolation

The `mesh_id` field in the registration schema supports multiple independent meshes. Cross-mesh peering policies and routing are deferred until demand materialises.

18.5 Index Service

When the number of registered bridges exceeds ~50, chain scanning for registry transactions may become slow. An optional index service could cache registry state for faster bootstrap.

18.6 BSV-20 Transfer Tracking

The current token indexing supports deploy and mint operations. Transfer tracking -- following token ownership as inscriptions move between outputs -- requires building a UTXO graph for ordinal positions. This is deferred to a future phase.

18.7 Full Block Fetching

Historical backfill currently relies on WhatsOnChain for block transaction lists. Adding `MSG_BLOCK` support behind a feature flag would enable fully self-sovereign backfill with no third-party dependency, at the cost of increased bandwidth and storage.

18.8 Rate Limiting and DDoS Protection

Per-IP rate limiting for unauthenticated API requests. Currently described in the security section but not implemented as a configurable module.

19. The Wood Wide Web

In the 1990s, forest ecologist Suzanne Simard discovered that trees are not individuals competing for resources. They are nodes in a network. Underground, fungal threads called mycelium connect every tree in the forest — routing nitrogen from alders to firs, sending sugar from old trees to seedlings, self-healing when threads are severed. Scientists call it the Wood Wide Web. It has been evolving for 1.5 billion years.

The parallels to the federation are not metaphorical convenience. They are structural convergence — the same engineering constraints produce the same architecture:

Forest	Federation
Mycelium threads	Bridges
Trees	Applications
Nutrients (nitrogen, sugar)	Transactions
Spore dispersal	DNS seed crawler
Mycorrhizal network (root-fungus interface)	REST API (app-bridge interface)
Fruiting bodies (mushrooms)	On-chain registry (visible discovery surface)
Soil taste (nutrient sensing)	Crawler bucket analysis (network health sensing)

No central hub — cut down the oldest tree, the network survives. Self-healing — sever one thread, nutrients reroute through others. Plug-and-play — new trees tap into existing infrastructure without building their own root system. Emergent cooperation without central control — the forest "decides" where resources go without a master controller.

The metaphor completed itself in April 2026. The legacy BSV DNS seeds — the spore banks of the ecosystem — went dark. Two domains stopped existing. Two more returned empty responses. The federation had been passively depending on external seed infrastructure. Like a forest island cut off from continental spore flow, the network risked slow atrophy as peers disconnected and no replacements arrived.

The response was to build the federation's reproductive system: a crawler that probes the BSV network every five minutes, tagging peers by failure mode, learning what's alive and what's ghost. A DNS seed at `seed.indelible.one` publishing verified-alive peers. Good peer persistence saving reliable peers to disk. The federation stopped depending on anything it could not replace.

The first crawler run revealed that 74% of "known" BSV peers were ghosts — IP addresses that used to run BSV nodes and now run something else entirely. The BSV network's self-image was polluted with the corpses of an older network. The crawler prunes the ghosts. It is how the mycelium forgets dead soil and remembers live soil.

Forests that survive a billion years are the ones that finish growing into themselves. They do not depend on anything they cannot replace.

20. Conclusion

The Indelible Federation transforms isolated SPV nodes into an open, self-governing network — a Wood Wide Web for Bitcoin. Any developer can run a bridge, register on-chain, and join the mesh — no permission required, no central authority, no API keys. The protocol uses BSV at every layer: identity (one keypair per bridge), registry (CBOR-encoded OP_RETURN transactions), security (stake bonds locked to the operator's own address), discovery (beacon address scanning, gossip, DNS seed crawler, and good peer persistence), trust (local peer scoring based on observed behaviour), and revenue (x402 micropayments for bridge operators).

Seven bridges operate across six geographic locations, managed by systemd, syncing 945,000+ block headers, relaying transactions with 52-69% novel relay hit rates, and serving three production applications. 422 tests verify the protocol implementation. The npm packages install with a single command.

The system is self-contained. It runs its own DNS seed. It runs its own crawler. It persists its own good peers. It discovers bridges through its own on-chain registry. No external dependency — no DNS seed, no API, no third-party service — can take this network down. Every bridge that joins makes the mesh stronger. Every application that connects creates demand for more bridges. Every bridge operator that earns x402 revenue has an incentive to keep their infrastructure running.

BSV has the technical capability — big blocks, cheap fees, native OP_RETURN, powerful script. What it has lacked is developer infrastructure. The federation fills the gap between "run a full node" and "depend on someone else's API." The mycelium is live. Just tap in.

References

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Available: <https://bitcoin.org/bitcoin.pdf>
 - [2] M. Hearn, M. Corallo, "BIP 37: Connection Bloom filtering," 2012. Available: <https://github.com/bitcoin/bips/blob/master/bip-0037.mediawiki>
 - [3] C. S. Wright, "Overlay Network Architecture for Bitcoin Scaling," SSRN Electronic Journal, SSRN-6277825, 2025. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6277825
 - [4] BRC-22, "Overlay Network Data Synchronization," BSV Blockchain Standards. Available: <https://github.com/bitcoin-sv/BRCs/blob/master/overlays/0022.md>
 - [5] BRC-33, "PeerServ Message Relay Interface," BSV Blockchain Standards. Available: <https://github.com/bitcoin-sv/BRCs/blob/master/peer-to-peer/0033.md>
 - [6] BRC-105, "HTTP Service Monetization," BSV Blockchain Standards. Available: <https://github.com/bitcoin-sv/BRCs/blob/master/payments/0105.md>
-

Indelible Federation — The Wood Wide Web for Bitcoin. Open infrastructure, secured by proof of work.

<https://github.com/zcoolz/relay-federation>